FEDERATED LEARNING FOR MEDICAL IMAGE CLASSIFICATION



FPT UNIVERSITY

Nguyen Quoc Bao
Tran Hoai Phuc
Ho Huu Huy
Nguyen Hoang Duy Khang

Supervisors: Assoc. Prof. Duc Ngoc Minh Dang
Dr. Tien Huu Do

Department of Artificial Intelligence
FPT University

This capstone project submitted in partial fulfillment of the requirement for the *Degree of Bachelor of Artificial Intelligence in Computer Science*

FPT University November 2026

Abstract

This is where you write your abstract ...

Table of contents

1 INTRODUCTION **1**

2 RELATED WORK **3**

2.1 Federated Learning in Healthcare and Medical Image 3

2.2 Edge Federated Learning and Hardware Heterogeneity 3

2.3 Addressing Statistical Heterogeneity (Non-IID Data) 4

2.4 Federated Learning Frameworks and Flower 4

3 METHODOLOGY **7**

3.1 Benchmark Reproduction and Experimental Foundation (FedSLD) 7

3.1.1 Benchmark Datasets: Focus on PathMNIST and OrganAMNIST . . 7

3.1.2 Data Partitioning: Practical versus Pathological Non-IID 8

3.1.3 Baseline Algorithms: FedAvg and FedProx 9

3.1.4 Experimental Configuration 9

3.1.5 Reproduction Results and Baseline Selection 10

3.2 Proposed Dual-Heterogeneity Method 12

3.2.1 Motivation and Dual Heterogeneity Challenges 12

3.2.2 Model Architecture: BatchNorm-Augmented CNN (570k Parameters) 13

3.2.3 Handling Domain Shift via Local Batch Normalization (FedBN) . . 15

3.2.4 Correcting Client Drift via SCAFFOLD with Zero-Drift Masking . 15

3.2.5 Federated Workflow and Lifecycle 17

4 EXPERIMENTAL AND RESULTS **19**

4.1 Datasets, Preprocessing, and Partitioning 19

4.1.1 Datasets and Data Preprocessing 19

4.1.2 Federated Non-IID Partitioning 20

4.2 Baselines 20

4.3 Main Experiments and Results 20

5 CONCLUSIONS **23**
5.1 First section of the third chapter 23
5.1.1 First subsection in the first section 23
5.1.2 Second subsection in the first section 23
5.1.3 Third subsection in the first section 23
5.2 Second section of the third chapter 24
5.3 The layout of formal tables 24

6 CHAPTER6 **27**
6.1 Conclusion 27
6.2 Future works 27

References **29**

Chapter 1

INTRODUCTION

Medical image classification is a fundamental task in computer-aided diagnosis, where deep learning models automatically assign disease or tissue categories to clinical images. Deploying accurate classification systems across hospitals and clinical centers has the potential to reduce diagnostic burden on clinicians, accelerate screening workflows, and improve early intervention rates. However, training such models typically requires large-scale, diverse imaging datasets aggregated from multiple institutions — a process that directly conflicts with patient privacy regulations such as Health Insurance Portability and Accountability Act (HIPAA) and General Data Protection Regulation (GDPR). Federated Learning (FL) offers a principled solution to this dilemma by enabling multiple clients — each representing a hospital, imaging lab, or edge device — to collaboratively train a shared global model while keeping all patient data local, communicating only model parameter updates to a central server.

Chapter 2

RELATED WORK

2.1 Federated Learning in Healthcare and Medical Image

In recent years, Federated Learning (FL) has emerged as a revolutionary approach in healthcare to solve the data silo problem while preserving patient privacy. Traditional machine learning requires centralizing large amounts of data, which is often legally and ethically impossible in the medical field due to strict privacy regulations such as HIPAA and GDPR. FL addresses this by allowing multiple medical institutions or devices to collaboratively train a shared model while keeping the raw data locally [1]. To accelerate research in this area, standardized and lightweight datasets, such as MedMNIST v2, have been introduced [2]. Datasets like BloodMNIST provide a diverse, standardized benchmark that allows researchers to easily evaluate the performance of classification algorithms on 2D medical images without requiring extensive clinical background knowledge [2].

2.2 Edge Federated Learning and Hardware Heterogeneity

While cross-silo FL involves powerful hospital servers, cross-device FL operates on resource-constrained edge devices [3]. Deploying deep learning models on embedded devices like the NVIDIA Jetson series introduces the challenge of system heterogeneity [4]. These edge devices vary significantly in compute capabilities, memory capacity, and power consumption. In a synchronous FL system, this hardware imbalance often leads to the straggler effect, where fast devices must idle and wait for the slowest devices to complete their local training rounds [3]. Research has demonstrated that evaluating FL on real heterogeneous hardware is

crucial for accurately measuring the trade-offs between model convergence time and overall energy consumption [4].

2.3 Addressing Statistical Heterogeneity (Non-IID Data)

A fundamental challenge in real-world FL is that local datasets across clients are rarely independent and identically distributed (Non-IID) [5]. The standard algorithm, Federated Averaging (FedAvg), often struggles in these environments due to a phenomenon known as client-drift [6]. Because each client's data distribution is different, local updates move towards the client's local optimum rather than the global optimum, leading to slow and unstable convergence [6].

To tackle this issue, several advanced algorithms have been proposed. The FedProx algorithm mitigates system and statistical heterogeneity by adding a proximal term to the local objective, which restricts local updates from drifting too far from the global model [7]. Alternatively, the SCAFFOLD algorithm uses variance reduction and control variates to explicitly estimate and correct this client-drift, allowing for faster convergence regardless of data heterogeneity [6]. In the specific context of medical imaging, data often suffers from feature shift—for example, images of cells or tissues can look different simply because they were captured by different hospital scanners [8]. To solve this, the FedBN algorithm proposes keeping Batch Normalization (BN) parameters strictly local on each client instead of averaging them on the server [8]. This lightweight modification has been proven to significantly harmonize local feature distributions and accelerate convergence in medical domains [8].

2.4 Federated Learning Frameworks and Flower

To transition FL from theoretical research to practical deployment on physical edge devices, flexible software frameworks are required. Many early FL frameworks only supported single-machine simulations or were restricted to specific machine learning libraries [4]. To bridge this gap, Flower (flwr) was developed as a scalable, framework-agnostic platform [4]. Flower allows researchers to seamlessly migrate workloads from simulated environments to a large cohort of real, heterogeneous edge devices [4]. By abstracting away the complex networking and communication protocols, Flower enables researchers to easily implement custom FL strategies (such as FedAvg, FedProx, FedBN, or Scaffold) and measure actual system performance metrics like memory usage and communication overhead [4].

Besides Flower, NVIDIA FLARE (NVFlare) [9] is another widely used federated learning platform. This study refer to NVFlare's implementation of SCAFFOLD to guide our own setup, particularly in how control variates are managed and exchanged between the server and clients.

Chapter 3

METHODOLOGY

3.1 Benchmark Reproduction and Experimental Foundation (FedSLD)

To establish a solid and fair baseline for our study, we begin by reproducing the benchmark from the FedSLD paper (Luo & Wu, ISBI 2022) [10]. FedSLD is a well-known federated learning study designed specifically for medical image classification under non-IID data distributions. In this section, we describe the datasets, the non-IID partitioning method, the experimental configuration, the foundational baseline algorithms (FedAvg and FedProx), and the reproduced benchmark results.

3.1.1 Benchmark Datasets: Focus on PathMNIST and OrganAMNIST

The MedMNIST v2 collection [2] provides standardized 2D biomedical image datasets for machine learning research. While the collection includes several datasets (such as BloodMNIST and DermaMNIST), this research focuses primarily on two representative benchmarks:

1. **PathMNIST**: A dataset of colon pathology images containing 9 diagnostic classes (such as adipose, debris, lymphocytes, and tumor epithelium). It exhibits significant variations in tissue textures and histological staining.

2. **OrganAMNIST**: A dataset of abdominal CT organ patches featuring 11 anatomical classes (such as liver, kidneys, spleen, and bladder). It presents subtle grayscale differences across diverse organs.

Other datasets in the collection, such as BloodMNIST (blood cell classification) and DermaMNIST (dermatoscopy skin lesion screening), are also supported in our codebase, but PathMNIST and OrganAMNIST serve as the core evaluation benchmarks in this thesis.

3.1.2 Data Partitioning: Practical versus Pathological Non-IID

In the FedSLD benchmark [10], the authors evaluate federated learning under two distinct non-IID partitioning schemes:

- **Pathological Non-IID**: Originally introduced in the foundational FedAvg paper [11], this classic scheme artificially restricts each client to hold samples from only a few classes (for example, exactly 2 classes per hospital). While it creates extreme label skew to test theoretical convergence, it is unrealistic in healthcare because actual hospitals treat a diverse spectrum of diseases, rather than just one or two specific conditions.
- **Practical Non-IID**: To overcome the artificial nature of the pathological split, FedSLD [10] designed the Practical partition, which reflects clinical reality much more accurately. In real-world healthcare collaborations, medical institutions differ substantially in size, patient volume, and diagnostic specialization.

Specifically, for $K = 12$ clients, the Practical partition applies clinical proportions **independently for each individual class**, rather than as a single global split across the entire dataset:

- For every diagnostic category, the samples belonging to that class are divided into 12 shards: one dominant shard containing **80%** of the class samples, one medium shard containing **10%**, and ten small shards sharing the remaining **10%** equally (**1%** each).
- These shards are then randomly assigned across the 12 clients for each class.

As a result of this per-class sharding mechanism, a single client can be assigned the 80% dominant shard for multiple disease categories. This accurately simulates large specialized medical centers that receive a high volume of cases across several related conditions (for instance, in PathMNIST, one large client holds the 80% dominant shards for three different classes, totaling over 26,000 samples). Meanwhile, other clients receive only small 1% shards across all classes (simulating small local clinics with under 900 samples). This partitioning strategy creates realistic volume disparities (a $\sim$29$\times$ sample count difference between the largest and smallest clients) alongside natural clinical specialization, while guaranteeing that all clients retain at least a baseline number of samples for every class.

3.1.3 Baseline Algorithms: FedAvg and FedProx

To evaluate model performance, we implement and reproduce two foundational federated learning algorithms:

1. **FedAvg (Federated Averaging)** [11]: The classic baseline algorithm. In each round, the central server sends the current global model to the clients. Each client trains the model on its local data using SGD and sends the updated weights back. The server then computes a weighted average of these weights based on each client's sample count:

$$w^{(r+1)} = \sum_{k=1}^{K} \frac{n_k}{N} w_k^{(r+1)} \tag{3.1}$$

   When data across clients is heavily non-IID, local models tend to move toward their local optima, causing the global model to suffer from *client drift*.

2. **FedProx (Federated Proximal)** [7]: An extension of FedAvg designed to improve stability under heterogeneity. FedProx adds a proximal regularization term to each client's local loss function:

$$\min_{w} \mathcal{L}_k(w) = F_k(w) + \frac{\mu}{2} \|w - w^{(r)}\|^2 \tag{3.2}$$

   where $\mu > 0$ is a proximal penalty parameter (we use $\mu = 0.01$). This term keeps local model updates closer to the global model $w^{(r)}$, helping to reduce client drift.

3.1.4 Experimental Configuration

To reproduce the FedSLD benchmark faithfully and ensure fair comparisons, we adopt the exact experimental parameters specified in the FedSLD paper:

- **Number of Clients**: $K = 12$ clients participating in federated training.
- **Communication Rounds**: 80 rounds.
- **Local Epochs**: 5 local epochs per client in each round.
- **Mini-Batch Size**: 256.
- **Optimizer**: Stochastic Gradient Descent (SGD) with momentum $\mu = 0.9$ and weight decay 10^{-4}.
- **Learning Rate**: Initial learning rate $\eta = 0.01$ (with a fixed schedule across rounds).

- **Network Architecture**: In their implementation details, Luo & Wu [10] specify: *"We use the classic four-layer CNN model with two* 5×5 *convolutional layers and two fully connected layers (hidden layer has 500 units)"*, following the standard federated benchmark model established by McMahan et al. [11]. This architecture comprises $\sim$570k parameters and operates without Batch Normalization layers.

Importantly, for our proposed method, we maintain this exact experimental configuration and strictly follow the partitioning design of the original paper—dividing each class into 12 shards according to the 80%–10%–1% clinical proportions. This guarantees that all subsequent performance comparisons and gains stem directly from algorithmic improvements rather than different hyper-parameters or altered partition settings.

3.1.5 Reproduction Results and Baseline Selection

Table 3.1 compares the original results published in the FedSLD paper [10] against our reproduction across both PathMNIST and OrganAMNIST under the Practical non-IID partition. For our reproduction, we report the mean and standard deviation across independent runs. We evaluate performance using the two official FedSLD metrics:

- **BMCTA (Best Mean Client Test Accuracy)**: The average test accuracy computed across each of the 12 individual client local test sets.
- **BTA (Best Total Accuracy)**: The overall test accuracy evaluated on the pooled global test set.

Table 3.1 Benchmark results under Practical Non-IID partition (12 clients, 80 rounds) compared with original FedSLD paper and Centralized Upper Bound.

Dataset	**Method / Algorithm**	**FedSLD Paper [10]**		**Our Reproduction**	
		BMCTA (%)	**BTA (%)**	**BMCTA (%)**	**BTA (%)**
OrganAMNIST	Centralized (Upper Bound)	—	—	81.99 ± 0.84	81.99 ± 0.84
	FedAvg	82.32	85.69	82.39 ± 0.84	82.98 ± 0.85
	FedProx	81.53	85.54	82.29 ± 0.81	82.88 ± 0.84
	FedSLD (Paper Method)	84.75	87.37	—	—
PathMNIST	Centralized (Upper Bound)	—	—	77.11 ± 1.33	77.11 ± 1.33
	FedAvg	52.70	57.38	77.76 ± 2.05	79.44 ± 1.26
	FedProx	52.77	57.72	77.99 ± 1.93	79.75 ± 0.76
	FedSLD (Paper Method)	53.87	57.90	—	—

Table 3.1 reveals several key empirical insights across the benchmarks:

1. **OrganAMNIST Consistency and Discrepancy**: On OrganAMNIST, our reproduced client-average test accuracy (BMCTA) matches the published paper remarkably well (82.39% vs. 82.32% for FedAvg). However, the global pooled test accuracy (BTA) achieved by our reproduction (82.98%) is approximately 2.7% lower than the value reported in the original paper (85.69%).

2. **PathMNIST Accuracy**: On PathMNIST, our reproduced models achieve substantially higher accuracy ($\approx 79.4\%$ BTA) than the relatively low numbers reported in the FedSLD paper ($\approx 57.4\%$ BTA).

3. **Federated vs. Centralized Gap**: As expected in non-IID federated learning, distributed training suffers from performance degradation compared to centralized training. The extreme sample imbalance in the Practical partition (where small clinics hold only 1% of samples per class) causes client drift that prevents FedAvg and FedProx from reaching centralized performance levels.

Discussion and Analysis of Reproduction Discrepancies. In experimental reports and scientific evaluations, addressing numerical differences between published literature and independent reproductions is essential. We attribute the observed performance gap on OrganAMNIST and PathMNIST to three fundamental factors:

- **Absence of Public Random Seeds**: The authors of FedSLD [10] did not release the exact random seed used for their dataset splitting or weight initialization. In the Practical non-IID partitioning scheme, each class is divided into 12 shards ($1\times80\%$, $1\times10\%$, $10\times1\%$) and randomly shuffled across clients. Because shard assignment is stochastic, different seeds produce drastically different client compositions. For example, if a random seed assigns the 80% dominant shards of several classes to a single client, that client becomes an overwhelming outlier, exacerbating client drift and lowering global convergence.

- **Non-Convex Optimization and Weight Initialization**: Deep neural networks trained in decentralized, non-IID settings are highly sensitive to initial parameter values. Different initializations cause local SGD trajectories to diverge into different local minima, leading to a natural variation of $2\% - 3\%$ across independent trials.

- **Unspecified Training Details**: Minor implementation nuances—such as mini-batch shuffling order, exact learning rate decay milestones, and gradient clipping thresholds—were not documented in the original publication, contributing to empirical variance.

Establishing Our Reproduction as the Fair Baseline. Rather than viewing these variations as a limitation, this discrepancy reinforces the necessity of establishing our own rigorous, fully documented benchmark. Directly comparing our subsequent proposed methodology against numbers from an un-seeded, non-reproducible external baseline would be scientifically invalid and unfair.

Therefore, we adopt our own reproduced results (evaluated over 5 independent runs with fixed random seeds and identical training pipelines) as the authoritative, consistent baseline. Any performance improvements demonstrated by our proposed Dual-Heterogeneity strategy in Chapter **??** are evaluated on this exact same data partition and baseline foundation, guaranteeing a strictly fair and credible comparison.

3.2 Proposed Dual-Heterogeneity Method

Having established a solid and reproducible baseline in Section 3.1, this section introduces our proposed federated learning framework. We design a unified strategy that addresses two major hurdles in decentralized healthcare AI: dual heterogeneity (feature shift and client drift) and edge resource constraints.

3.2.1 Motivation and Dual Heterogeneity Challenges

In real-world medical federated networks, data heterogeneity manifests in two distinct, orthogonal dimensions:

1. **Feature/Domain Shift**: Different hospitals and medical centers utilize varying imaging equipment, scanning protocols, radiation dosages, and histological staining techniques. As a result, the input features (such as pixel intensities, texture patterns, and color distributions) differ across institutions even for identical anatomical structures or disease conditions.

2. **Client Drift from Label and Volume Imbalance**: Participating clinical sites naturally differ in patient volume and medical specialization. In the Practical non-IID partition (as detailed in Section 3.1), a large medical center may hold 80% of the cases for several disease categories, while smaller clinics hold only 1% shards. Under standard federated averaging (FedAvg), local model updates from smaller or specialized clients quickly drift toward their local optima, causing destructive interference during global aggregation.

Existing methods typically address only one of these two issues:

- **FedBN** [8] effectively mitigates feature shift by preserving local Batch Normalization layers, but it does not prevent client drift caused by extreme label distribution skew.
- **SCAFFOLD** [6] corrects client drift using control variates, but it assumes that all clients share identical feature representations and does not account for domain-level scanner variations.

Moreover, the classic baseline CNN architecture used in the original FedSLD paper [10] ($\sim$570k parameters) does not incorporate any normalization layers. Without normalization mechanisms, it is fundamentally impossible to isolate client-specific scanner variations from shared semantic representations via techniques like FedBN.

To overcome these combined bottlenecks while maintaining strict comparability with the baseline, we propose a synergistic framework that augments the **classic 4-layer CNN architecture ($\sim$570k parameters) with Batch Normalization layers** and applies a **Dual-Heterogeneity strategy (FedBN + SCAFFOLD)** featuring **Zero-Drift Masking** and **decoupled post-step updates**.

3.2.2 Model Architecture: BatchNorm-Augmented CNN (570k Parameters)

To ensure an exact and fair comparison with the FedSLD baseline, our proposed framework preserves the identical backbone capacity and layer dimensions of the classic 4-layer CNN, but integrates Batch Normalization (`BatchNorm2d`) immediately following each convolutional layer.

The detailed layer specifications of our BatchNorm-augmented model (`BaselineBN`) are:

1. **Convolutional Block 1**: A 5×5 2D convolution layer with 32 filters (stride = 1, padding = 0), followed by Batch Normalization (`BatchNorm2d`(32)), a ReLU activation function, and a 2×2 Max-Pooling layer with stride 2. This reduces the spatial resolution from $28 \times 28 \rightarrow 24 \times 24 \rightarrow 12 \times 12$.
2. **Convolutional Block 2**: A 5×5 convolution increasing channels from 32 to 64 (stride = 1, padding = 0), followed by `BatchNorm2d`(64), ReLU, and 2×2 Max-Pooling (stride 2), reducing spatial size from $12 \times 12 \rightarrow 4 \times 4$.
3. **Dense Hidden Layer**: The resulting $64 \times 4 \times 4 = 1024$ flattened tensor is projected through a fully connected layer with 500 hidden units (Linear$(1024, 500)$), followed by a ReLU activation.

4. **Classification Head**: A final linear projection layer mapping the 500 hidden features directly to class logits (Linear$(500, C)$, where C is the number of classes, e.g., 9 for PathMNIST and 11 for OrganAMNIST).

Table 3.2 and Fig. 3.1 present a direct structural comparison between the un-normalized FedSLD baseline CNN and our BatchNorm-augmented CNN.

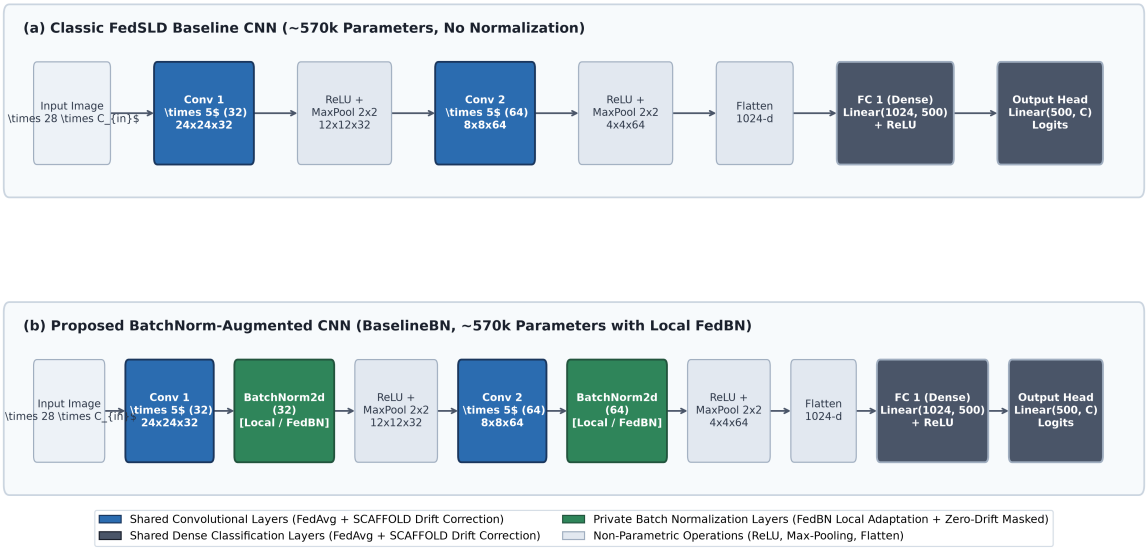


Fig. 3.1 Architectural comparison between (a) the classic un-normalized 4-layer CNN from the FedSLD paper [10], and (b) our proposed BatchNorm-augmented CNN (`BaselineBN`). In our proposed method, the convolutional filters and dense classification layers (blue and dark gray) are aggregated globally with SCAFFOLD drift correction, whereas the Batch Normalization layers (green) are kept strictly local to each client hospital (FedBN) and masked from drift correction ($M_l = 0$).

Table 3.2 Architectural comparison between classic baseline CNN and our BatchNorm-augmented model.

Layer / Component	**Baseline CNN [10]**	**Proposed Model (BaselineBN)**
Conv Block 1	5×5 Conv, 32 + ReLU + Pool	5×5 Conv, 32 + **BN** + ReLU + Pool
Conv Block 2	5×5 Conv, 64 + ReLU + Pool	5×5 Conv, 64 + **BN** + ReLU + Pool
Dense Layer	Linear$(1024 \to 500)$ + ReLU	Linear$(1024 \to 500)$ + ReLU
Output Layer	Linear$(500 \to C)$	Linear$(500 \to C)$
Total Parameters	$\sim$ 570k (570, 705 on PathMNIST)	$\sim$ 570k (570, 897 on PathMNIST)
Parameter Delta	Baseline (0 BN params)	$+192$ BN params ($< 0.04\%$)

As shown in Table 3.2, adding Batch Normalization introduces merely 192 additional parameters (comprising affine weights and biases for the 32 and 64 feature maps), representing

less than a 0.04% change in total model size. Consequently, the feature representation capacity and memory footprint remain virtually identical to the original FedSLD baseline ($\sim$570k parameters). This design ensures that any performance improvements achieved by our proposed method arise strictly from algorithmic advancements (FedBN domain adaptation and SCAFFOLD drift correction) rather than differences in model scale.

3.2.3 Handling Domain Shift via Local Batch Normalization (FedBN)

Batch Normalization layers standardize layer inputs using the mean μ_B and variance σ_B^2 computed across mini-batch B, followed by learnable affine parameters (scale γ and shift β):

$$\hat{x} = \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \quad y = \gamma\hat{x} + \beta \tag{3.3}$$

During training, the layer also maintains exponential running statistics ($\mu_{\text{run}}, \sigma^2_{\text{run}}$) to approximate the global feature distribution for inference.

In standard federated aggregation (such as FedAvg), all parameters—including Batch Normalization statistics—are averaged across clients. However, when different hospitals use different scanners and staining techniques, averaging these parameters forces every client to adopt a blurred, contradictory feature representation, degrading local accuracy.

Following the principles of FedBN [8], our framework partitions the model parameters into two distinct subsets:

1. **Global Shared Parameters** (Θ_{global}): The weights and biases of convolutional filters and the final classification head ($W_{\text{conv}}, W_{\text{fc}}$). These capture general visual representations and semantic category boundaries, and are aggregated by the central server.

2. **Local Private Parameters** (Θ_{local}): All Batch Normalization parameters (γ, β) and running statistics ($\mu_{\text{run}}, \sigma^2_{\text{run}}$). These parameters remain strictly on each local client and are never uploaded to the server or averaged.

By keeping Batch Normalization local, each hospital client standardizes internal activations according to its specific imaging distribution, harmonizing domain differences while collaboratively benefiting from shared representation learning.

3.2.4 Correcting Client Drift via SCAFFOLD with Zero-Drift Masking

While local Batch Normalization resolves domain discrepancies, it does not alleviate the severe gradient bias induced by label and sample volume disparities. To prevent local models

from drifting away from the global optimum, our framework incorporates control variates based on SCAFFOLD [6], enhanced with two critical adaptations: **Zero-Drift Masking** and **Decoupled Post-Step Updates**.

Control Variate Formulation

In SCAFFOLD, the server maintains a global control variate c, and each client i maintains a local control variate c_i. The control variate c_i estimates the average update direction of client i, while c represents the consensus update direction across the federation.

During local training, client i corrects its local gradient $g_i(w)$ by applying the drift difference $(c - c_i)$:

$$\tilde{g}_i(w) = g_i(w) - c_i + c \tag{3.4}$$

If client i receives a disproportionate number of samples from a single class, its local gradient $g_i(w)$ pulls the weights strongly toward that class. The term $-c_i + c$ counteracts this local bias and steers the trajectory back toward the global objective.

Zero-Drift Masking for BatchNorm Layers

A key technical novelty in our framework is the interaction between FedBN and SCAFFOLD. Because Batch Normalization parameters are deliberately kept private to preserve client-specific domain adaptation, applying global control variate correction to BN parameters would counteract their local specialization.

To maintain algorithmic harmony, we implement a **Zero-Drift Mask** $M \in \{0, 1\}^{|\Theta|}$:

$$M_l = \begin{cases} 0, & \text{if parameter } l \text{ belongs to a BatchNorm layer,} \\ 1, & \text{otherwise (convolutional and linear weights).} \end{cases} \tag{3.5}$$

The effective drift correction applied to layer l is governed by:

$$\Delta_{\text{drift},l} = M_l \cdot (c_l - c_{i,l}) \tag{3.6}$$

This ensures that non-BN layers receive full drift correction across heterogeneous label distributions, while BN layers remain unperturbed to adapt freely to local scanner characteristics.

Decoupled Post-Step Parameter Update

In standard federated implementations, the correction term is often added directly into the gradient tensor prior to optimizer stepping ($g \leftarrow g + c - c_i$). However, as observed in production frameworks such as NVIDIA FLARE [9], injecting drift directly into the gradient corrupts the momentum velocity buffers in SGD (when momentum $\beta > 0$) or the first and second moment vectors in Adam. Over multiple local epochs, this momentum pollution leads to numerical instability and divergence.

To resolve this issue, we decouple the drift correction from the optimizer. For each mini-batch step, client i first performs standard backpropagation and optimizer updating:

$$w \leftarrow \text{OptimizerStep}\Big(w,\ \nabla \mathcal{L}(w; \mathcal{B})\Big) \tag{3.7}$$

Immediately following the optimizer step, the masked drift correction is applied directly to the parameter tensor without gradient tracking:

$$w \leftarrow w - \eta \cdot \Delta_{\text{drift}} \tag{3.8}$$

where η is the current local learning rate. This decoupled update guarantees exact mathematical drift compensation while maintaining clean momentum buffers across varying optimizers and learning rate schedules.

3.2.5 Federated Workflow and Lifecycle

The complete execution lifecycle of our proposed framework across communication rounds proceeds through the following sequential stages:

1. **Server Broadcast (Round r)**: The central server broadcasts the current global model weights $w_{\text{global}}^{(r)}$ (covering non-BN parameters) and the global control variate $c^{(r)}$ to all participating clients $i \in \{1, \ldots, K\}$.

2. **Local Initialization**: Client i receives $w_{\text{global}}^{(r)}$ and loads these weights into its local model while retaining its local Batch Normalization parameters ($\gamma_i, \beta_i, \mu_{i,\text{run}}, \sigma^2_{i,\text{run}}$).

3. **Local Training with Decoupled Drift**: Client i trains for E local epochs over its private dataset $\mathcal{D}_i$. For each mini-batch, the client executes the standard optimizer step and immediately applies the masked drift update $-\eta \cdot M \cdot (c^{(r)} - c_i^{(r)})$.

4. **Local Control Variate Update**: At the conclusion of local training, client i computes its updated local control variate $c_i^{(r+1)}$ following SCAFFOLD Option 2:

$$c_i^{(r+1)} = c_i^{(r)} - c^{(r)} + \frac{1}{K_{\text{steps}} \cdot \eta}\left(w_{\text{global}}^{(r)} - w_i^{(r+1)}\right) \tag{3.9}$$

   where K_{steps} is the total number of local gradient steps taken. The client then computes the control variate delta $\Delta c_i = c_i^{(r+1)} - c_i^{(r)}$.

5. **Client Upload**: Client i sends only its updated trainable non-BN weights $w_i^{(r+1)}$ and control variate delta Δc_i back to the server. Local BN parameters remain private on the device.

6. **Server Aggregation**: The server aggregates the received non-BN parameters using sample-weighted averaging, and updates the global control variate:

$$w_{\text{global}}^{(r+1)} = \sum_{i=1}^{K} \frac{n_i}{N} w_i^{(r+1)} \tag{3.10}$$

$$c^{(r+1)} = c^{(r)} + \frac{1}{K} \sum_{i=1}^{K} \Delta c_i \tag{3.11}$$

   where $n_i = |\mathcal{D}_i|$ and $N = \sum_{i=1}^{K} n_i$.

By unifying the standard $\sim$570k CNN architecture with private local Batch Normalization (FedBN) and masked decoupled control variates (SCAFFOLD), our proposed framework simultaneously addresses domain shifts and client drift without altering the fundamental model scale. Detailed empirical validations, comparisons against other FL strategies, and ablation studies are presented in Chapter **??**.

Chapter 4

EXPERIMENTAL AND RESULTS

4.1 Datasets, Preprocessing, and Partitioning

4.1.1 Datasets and Data Preprocessing

To evaluate the proposed framework across heterogeneous clinical distributions, we benchmark on three representative datasets from the MedMNIST v2 [2] standardized collection: PathMNIST, BloodMNIST, and OrganAMNIST. These benchmarks span four fundamental medical imaging modalities: histopathology slides, optical blood smear microscopy, volumetric abdominal computed tomography (CT), and surface dermatoscopy.

Table 4.1 Summary of benchmark medical imaging datasets used in our federated evaluation.

Benchmark	**Clinical Modality**	**Classes**	**Train**	**Val**	**Test**	**Total**
PathMNIST	Colon Histopathology	9	89,996	10,004	7,180	107,180
BloodMNIST	Blood Cell Microscopy	8	11,959	1,712	3,421	17,092
OrganAMNIST	Abdominal CT	11	34,561	6,491	17,778	58,830

All images are standardized to a spatial resolution of 28×28 pixels. To ensure structural compatibility across models without modifying initial convolution layers, single-channel grayscale CT slices from OrganAMNIST are projected to three identical channels via channel replication ($\mathbf{x} \in \mathbb{R}^{3\times28\times28}$). Pixel intensities in $[0, 1]$ are linearly normalized to $[-1, 1]$ using channel-wise statistics ($\mu = 0.5, \sigma = 0.5$).

4.1.2 Federated Non-IID Partitioning

Addressing severe data heterogeneity across healthcare institutions, we evaluate algorithmic resilience by partitioning datasets across $K = 12$ federated clients using two non-IID regimes: (i)Practical clinical partitioning and (ii) Dirichlet distribution partitioning.

4.2 Baselines

and here I write more …

4.3 Main Experiments and Results

This section has been modified from "Publication quality tables in LaTeX*" by Simon Fear.

The layout of a table has been established over centuries of experience and should only be altered in extraordinary circumstances.

When formatting a table, remember two simple guidelines at all times:

1. Never, ever use vertical rules (lines).
2. Never use double rules.

These guidelines may seem extreme but I have never found a good argument in favour of breaking them. For example, if you feel that the information in the left half of a table is so different from that on the right that it needs to be separated by a vertical line, then you should use two tables instead. Not everyone follows the second guideline:

There are three further guidelines worth mentioning here as they are generally not known outside the circle of professional typesetters and subeditors:

3. Put the units in the column heading (not in the body of the table).
4. Always precede a decimal point by a digit; thus 0.1 *not* just .1.
5. Do not use 'ditto' signs or any other such convention to repeat a previous value. In many circumstances a blank will serve just as well. If it won't, then repeat the value.

A frequently seen mistake is to use '\begin{center}' …'\end{center}' inside a figure or table environment. This center environment can cause additional vertical space. If you want to avoid that just use '\centering'

Table 4.2 A badly formatted table

	Species I		Species II	
Dental measurement	mean	SD	mean	SD
I1MD	6.23	0.91	5.2	0.7
I1LL	7.48	0.56	8.7	0.71
I2MD	3.99	0.63	4.22	0.54
I2LL	6.81	0.02	6.66	0.01
CMD	13.47	0.09	10.55	0.05
CBL	11.88	0.05	13.11	0.04

Table 4.3 A nice looking table

Dental measurement	Species I		Species II	
	mean	SD	mean	SD
I1MD	6.23	0.91	5.2	0.7
I1LL	7.48	0.56	8.7	0.71
I2MD	3.99	0.63	4.22	0.54
I2LL	6.81	0.02	6.66	0.01
CMD	13.47	0.09	10.55	0.05
CBL	11.88	0.05	13.11	0.04

Table 4.4 Even better looking table using booktabs

Dental measurement	Species I		Species II	
	mean	SD	mean	SD
I1MD	6.23	0.91	5.2	0.7
I1LL	7.48	0.56	8.7	0.71
I2MD	3.99	0.63	4.22	0.54
I2LL	6.81	0.02	6.66	0.01
CMD	13.47	0.09	10.55	0.05
CBL	11.88	0.05	13.11	0.04

Chapter 5

CONCLUSIONS

5.1 First section of the third chapter

And now I begin my third chapter here …
And now to cite some more people **? ?**]

5.1.1 First subsection in the first section

…and some more

5.1.2 Second subsection in the first section

…and some more …

First subsub section in the second subsection

…and some more in the first subsub section otherwise it all looks the same doesn't it? well we can add some text to it …

5.1.3 Third subsection in the first section

…and some more …

First subsub section in the third subsection

…and some more in the first subsub section otherwise it all looks the same doesn't it? well we can add some text to it and some more and some more and some more and some more and some more and some more and some more …

Second subsub section in the third subsection

…and some more in the first subsub section otherwise it all looks the same doesn't it? well we can add some text to it …

5.2 Second section of the third chapter

and here I write more …

5.3 The layout of formal tables

This section has been modified from "Publication quality tables in LaTeX*" by Simon Fear.

The layout of a table has been established over centuries of experience and should only be altered in extraordinary circumstances.

When formatting a table, remember two simple guidelines at all times:

1. Never, ever use vertical rules (lines).
2. Never use double rules.

These guidelines may seem extreme but I have never found a good argument in favour of breaking them. For example, if you feel that the information in the left half of a table is so different from that on the right that it needs to be separated by a vertical line, then you should use two tables instead. Not everyone follows the second guideline:

There are three further guidelines worth mentioning here as they are generally not known outside the circle of professional typesetters and subeditors:

3. Put the units in the column heading (not in the body of the table).
4. Always precede a decimal point by a digit; thus 0.1 *not* just .1.
5. Do not use 'ditto' signs or any other such convention to repeat a previous value. In many circumstances a blank will serve just as well. If it won't, then repeat the value.

A frequently seen mistake is to use '\begin{center}' …'\end{center}' inside a figure or table environment. This center environment can cause additional vertical space. If you want to avoid that just use '\centering'

Table 5.1 A badly formatted table

	Species I		Species II	
Dental measurement	mean	SD	mean	SD
I1MD	6.23	0.91	5.2	0.7
I1LL	7.48	0.56	8.7	0.71
I2MD	3.99	0.63	4.22	0.54
I2LL	6.81	0.02	6.66	0.01
CMD	13.47	0.09	10.55	0.05
CBL	11.88	0.05	13.11	0.04

Table 5.2 A nice looking table

Dental measurement	Species I		Species II	
	mean	SD	mean	SD
I1MD	6.23	0.91	5.2	0.7
I1LL	7.48	0.56	8.7	0.71
I2MD	3.99	0.63	4.22	0.54
I2LL	6.81	0.02	6.66	0.01
CMD	13.47	0.09	10.55	0.05
CBL	11.88	0.05	13.11	0.04

Table 5.3 Even better looking table using booktabs

Dental measurement	Species I		Species II	
	mean	SD	mean	SD
I1MD	6.23	0.91	5.2	0.7
I1LL	7.48	0.56	8.7	0.71
I2MD	3.99	0.63	4.22	0.54
I2LL	6.81	0.02	6.66	0.01
CMD	13.47	0.09	10.55	0.05
CBL	11.88	0.05	13.11	0.04

Chapter 6

CHAPTER6

6.1 Conclusion

6.2 Future works

References

[1] T. T. Le, P.-N. Tran, N. T. Pham, B. Manavalan, L. Shen, C. S. Hong, and D. N. M. Dang, "Federated learning for medical image analysis: Methods, challenges, and future directions," *Advanced Engineering Informatics*, vol. 76, p. 104976, 2026.

[2] J. Yang, R. Shi, D. Wei, Z. Liu, L. Zhao, B. Ke, H. Pfister, and B. Ni, "Medmnist v2-a large-scale lightweight benchmark for 2d and 3d biomedical image classification," *Scientific Data*, vol. 10, no. 1, p. 41, 2023.

[3] P. Kairouz, H. B. McMahan, B. Avent, A. Bellet, M. Bennis, A. N. Bhagoji, K. Bonawitz, Z. Charles, G. Cormode, R. Cummings *et al.*, "Advances and open problems in federated learning," *Foundations and Trends® in Machine Learning*, vol. 14, no. 1–2, pp. 1–210, 2021.

[4] D. J. Beutel, T. Topal, A. Mathur, X. Qiu, J. Fernandez-Marques, Y. Gao, L. Sani, K. H. Li, T. Parcollet, P. P. B. de Gusmão *et al.*, "Flower: A friendly federated learning research framework," *arXiv preprint arXiv:2007.14390*, 2020.

[5] D. M. Jimenez G, D. Solans, M. A. Heikkilä, A. Vitaletti, N. Kourtellis, A. Anagnostopoulos, and I. Chatzigiannakis, "Non-iid data in federated learning: A survey with taxonomy, metrics, methods, frameworks and future directions," *IEEE Communications Surveys & Tutorials*, 2024.

[6] S. P. Karimireddy, S. Kale, M. Mohri, S. Reddi, S. Stich, and A. T. Suresh, "Scaffold: Stochastic controlled averaging for federated learning," in *International Conference on Machine Learning*. PMLR, 2020, pp. 5132–5143.

[7] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, "Federated optimization in heterogeneous networks," in *Proceedings of Machine Learning and Systems*, vol. 2, 2020, pp. 429–450.

[8] X. Li, M. Jiang, X. Zhang, M. Kamp, and Q. Dou, "Federated learning on non-iid features via local batch normalization," in *International Conference on Learning Representations*, 2021.

[9] H. R. Roth, Y. Cheng, Y. Yang, I. Gao, Z. Shen, C. Lu, M. Harouni, Z. Xu, D. Zhao *et al.*, "Nvidia flare: Federated learning application runtime environment," *arXiv preprint arXiv:2210.13291*, 2022.

[10] J. Luo and M. Wu, "Fedsld: Federated learning with shared label distribution for medical image classification," in *IEEE International Symposium on Biomedical Imaging (ISBI)*. IEEE, 2022, pp. 1–5.

[11] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Artificial intelligence and statistics*. PMLR, 2017, pp. 1273–1282.